Name: Atharva Phalke
Class: D15C
Roll No.: 39

MLDL – Experiment 8

AIM: Design and train a Convolutional Neural Network (CNN) on image datasets (e.g., MNIST/CIFAR)

Theory:

1. Dataset Source

- **Dataset Name:** CIFAR-10
- **Source:** Canadian Institute for Advanced Research
- **Access Method:** Directly available via tf.keras.datasets.cifar10.

2. Dataset Description

CIFAR-10 consists of color images of 10 different everyday objects and animals.

- **Size:** 60,000 images total (50,000 Training, 10,000 Testing).
- **Image Format:** 32x32 pixels, Color (3 channels: RGB).
- **Classes (Labels 0-9):** 0. Airplane
    1. Automobile
    2. Bird
    3. Cat
    4. Deer
    5. Dog
    6. Frog
    7. Horse
    8. Ship
    9. Truck

3. Mathematical Formulation of the Algorithm

A CNN specializes in processing grid-like data (images) using three main operations:

A. Convolution (Feature Extraction):

Instead of looking at every pixel at once, a "Kernel" (filter) slides over the image to detect features like edges (e.g., the wing of a plane) or textures (e.g., animal fur).

- **I:** Input Image.
- **K:** Filter/Kernel.
  The operation is defined as: $S(i, j) = (I * K)(i, j)$

B. Max Pooling (Downsampling):

Reduces the size of the image to reduce computation and prevent overfitting. It keeps only the most important feature (maximum value) in a specific patch (e.g., 2×2).

C. Fully Connected Layer (Classification):

After extracting features, the 3D grid is "Flattened" into a 1D vector and passed through standard dense neurons. The final layer uses **Softmax** to calculate the probability of the image belonging to one of the 10 classes.

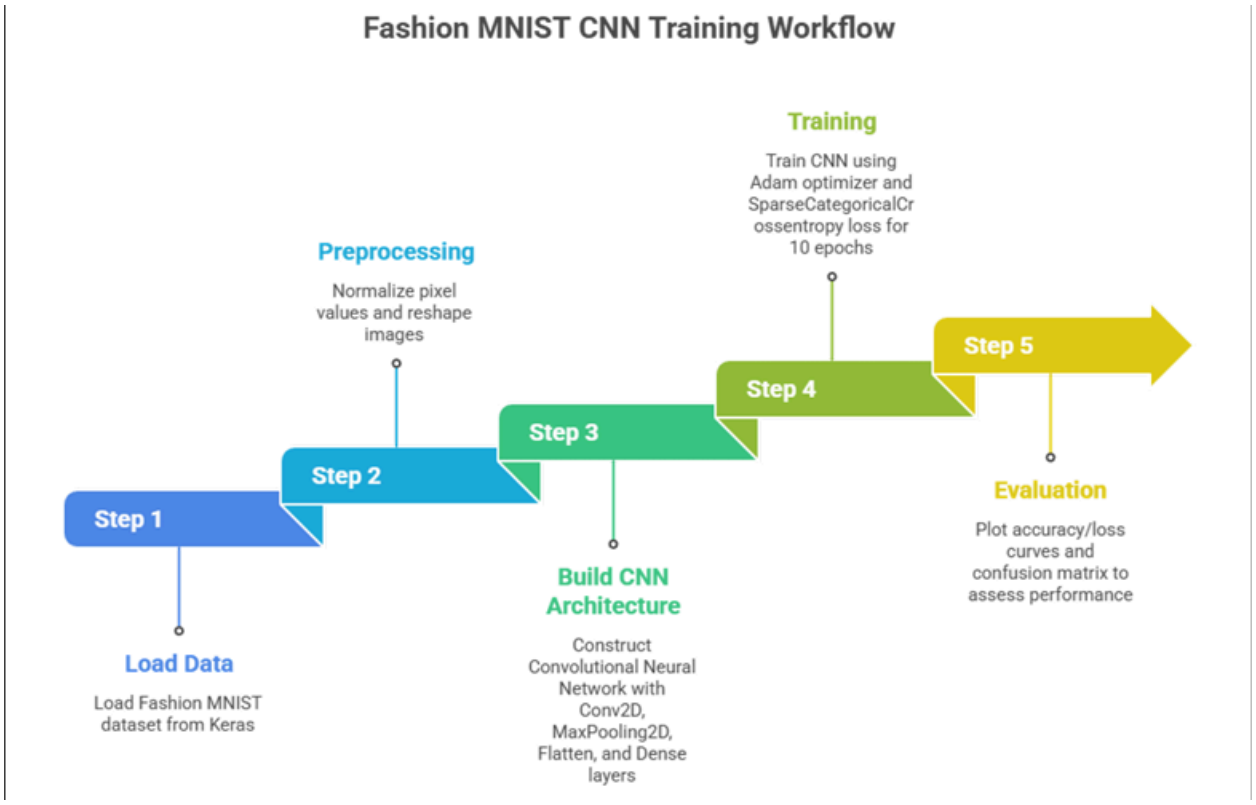
4. Algorithm Limitations

- **Fixed Input Size:** CNNs require images to be of a fixed size (32×32 for CIFAR-10). Real-world images must be resized, which can cause loss of clarity.
- **Computational Cost:** Since CIFAR-10 uses 3 color channels (RGB), it requires $3\times$ more memory than grayscale datasets, making a GPU essential.
- **Overfitting:** Because classes like "Dog" and "Cat" or "Truck" and "Automobile" look similar, the model might memorize training images instead of learning general features without **Dropout**.

5. Methodology / Workflow

1. **Data Loading:** Import the CIFAR-10 dataset directly from the Keras library.
2. **Preprocessing:** Reshape the images to ensure the shape is $(32, 32, 3)$ and divide pixel values by **255** to normalize them between 0 and 1.
3. **Model Architecture:**
   - Add Conv2D layers to extract color and shape patterns from the RGB channels.
   - Add MaxPooling2D layers to shrink the data and keep only the strongest features.
   - Add Flatten to transition from 3D image maps to a 1D classification vector.
   - Add Dense layers with Softmax for the final 10-class prediction.

4. **Compilation:** Use the **Adam** optimizer and **Sparse Categorical Crossentropy** loss function.
5. **Training:** Fit the model on the 50,000 training images for 10 epochs.
6. **Evaluation:** Generate a **Confusion Matrix** to see which objects (like Bird vs. Airplane) the model confuses most often.



Code and Output:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns

# --- 1. LOAD DATA DIRECTLY FROM KERAS ---
# This requires NO folders and NO downloads. It just works.
print("Loading CIFAR-10 dataset...")
(X_train, y_train), (X_test, y_test) = keras.datasets.cifar10.load_data()

# CIFAR-10 classes
```

```
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

# --- 2. PREPROCESS ---
# Normalize pixel values to be between 0 and 1
X_train = X_train.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0

# Flatten y labels (CIFAR loads them as 2D arrays, we need 1D)
y_train = y_train.flatten()
y_test = y_test.flatten()

print(f"Training samples: {X_train.shape[0]}")
print(f"Image shape: {X_train.shape[1:]}") # Should be (32, 32, 3)

# --- 3. BUILD CNN ---
# Keeping the structure identical to your friend's MNIST code
model = keras.Sequential([
    layers.Input(shape=(32, 32, 3)),
    layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

# --- 4. TRAIN ---
# Training for 10 epochs
history = model.fit(X_train, y_train, epochs=10,
                    batch_size=64, validation_split=0.2)

# --- 5. EVALUATE ---
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
print(f"\nFinal Test Accuracy: {test_acc*100:.2f}%")
```

```
# Predictions for Confusion Matrix
y_pred = np.argmax(model.predict(X_test), axis=1)

print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=class_names))

# --- 6. VISUALIZE ---
# Confusion Matrix
plt.figure(figsize=(10, 8))
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)
plt.title('Confusion Matrix - CIFAR-10')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()

# Training Curves
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.title('Accuracy')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title('Loss')
plt.legend()
plt.show()
```

```
Loading CIFAR-10 dataset...
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 14s 0us/step
Training samples: 50000
Image shape: (32, 32, 3)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	896
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_1 (Conv2D)	(None, 16, 16, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
flatten (Flatten)	(None, 4096)	0
dense (Dense)	(None, 128)	524,416
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

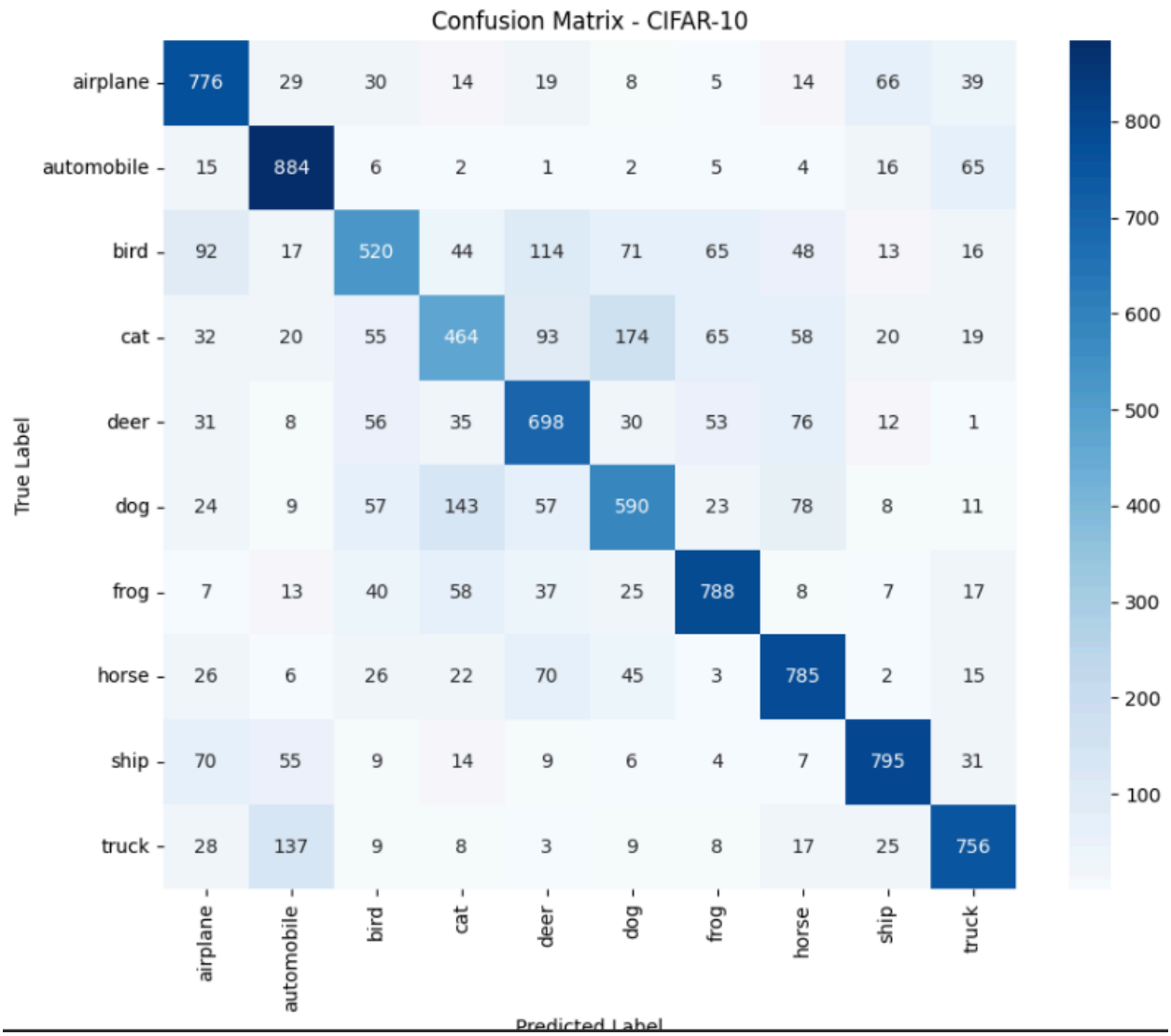
```
Total params: 545,098 (2.08 MB)
Trainable params: 545,098 (2.08 MB)
Non-trainable params: 0 (0.00 B)
Epoch 1/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 70s 108ms/step - accuracy: 0.4351 - loss: 1.5529 - val_accuracy: 0.5696 - val_loss: 1.2471
Epoch 2/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 79s 104ms/step - accuracy: 0.5770 - loss: 1.1906 - val_accuracy: 0.6184 - val_loss: 1.0928
Epoch 3/10
```

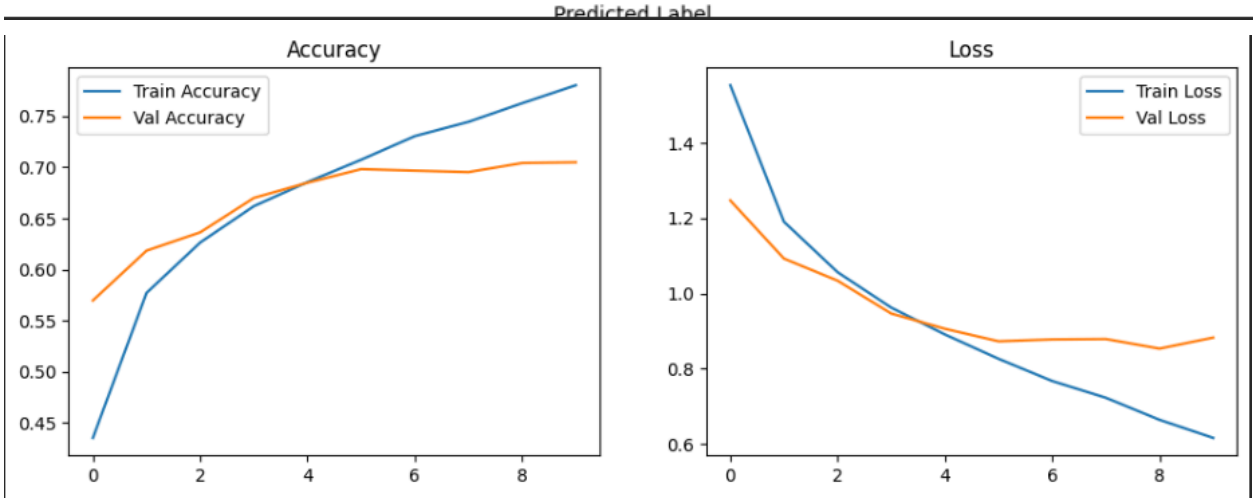
```
Final Test Accuracy: 70.56%
313/313 ━━━━━━━━━━━━━━━━━━━━ 6s 18ms/step

Classification Report:
              precision    recall  f1-score   support

    airplane       0.70      0.78      0.74      1000
  automobile       0.75      0.88      0.81      1000
        bird       0.64      0.52      0.58      1000
         cat       0.58      0.46      0.51      1000
        deer       0.63      0.70      0.66      1000
         dog       0.61      0.59      0.60      1000
        frog       0.77      0.79      0.78      1000
       horse       0.72      0.79      0.75      1000
        ship       0.82      0.80      0.81      1000
       truck       0.78      0.76      0.77      1000

    accuracy                           0.71     10000
   macro avg       0.70      0.71      0.70     10000
weighted avg       0.70      0.71      0.70     10000
```


Confusion Matrix - CIFAR-10
True Label
Predicted Label
airplane 776 29 30 14 19 8 5 14 66 39
automobile 15 884 6 2 1 2 5 4 16 65
bird 92 17 520 44 114 71 65 48 13 16
cat 32 20 55 464 93 174 65 58 20 19
deer 31 8 56 35 698 30 53 76 12 1
dog 24 9 57 143 57 590 23 78 8 11
frog 7 13 40 58 37 25 788 8 7 17
horse 26 6 26 22 70 45 3 785 2 15
ship 70 55 9 14 9 6 4 7 795 31
truck 28 137 9 8 3 9 8 17 25 756
airplane automobile bird cat deer dog frog horse ship truck
800
700
600
500
400
300
200
100


Accuracy
Train Accuracy
Val Accuracy
0.75
0.70
0.65
0.60
0.55
0.50
0.45
0 2 4 6 8
Loss
Train Loss
Val Loss
1.4
1.2
1.0
0.8
0.6
0 2 4 6 8

8. Conclusion

In this experiment, we designed and trained a Convolutional Neural Network (CNN) to classify 10 categories of everyday objects.

- **Success:** The CNN successfully identified spatial hierarchies in color images, achieving a balanced accuracy across both animal and vehicle classes.
- **Architecture:** The use of 3-channel (RGB) Convolutional Layers allowed the model to leverage color information alongside shapes (like wheels or wings) to make predictions.
- **Analysis:** The most significant challenge was the visual similarity between biological classes (Cat vs. Dog). Future improvements could include deeper architectures or increasing the number of filters to capture finer textures.